

Stagiaire : André Mazères

Durée du stage : 5 semaines

Lieu : M.A.Y.A.L, 35 rue Carnot ,92150 Suresnes, France

Projet : Mise en place d'un serveur Linux, supervision Prometheus/Grafana, VPN ZeroTier/WireGuard, automatisation PowerShell

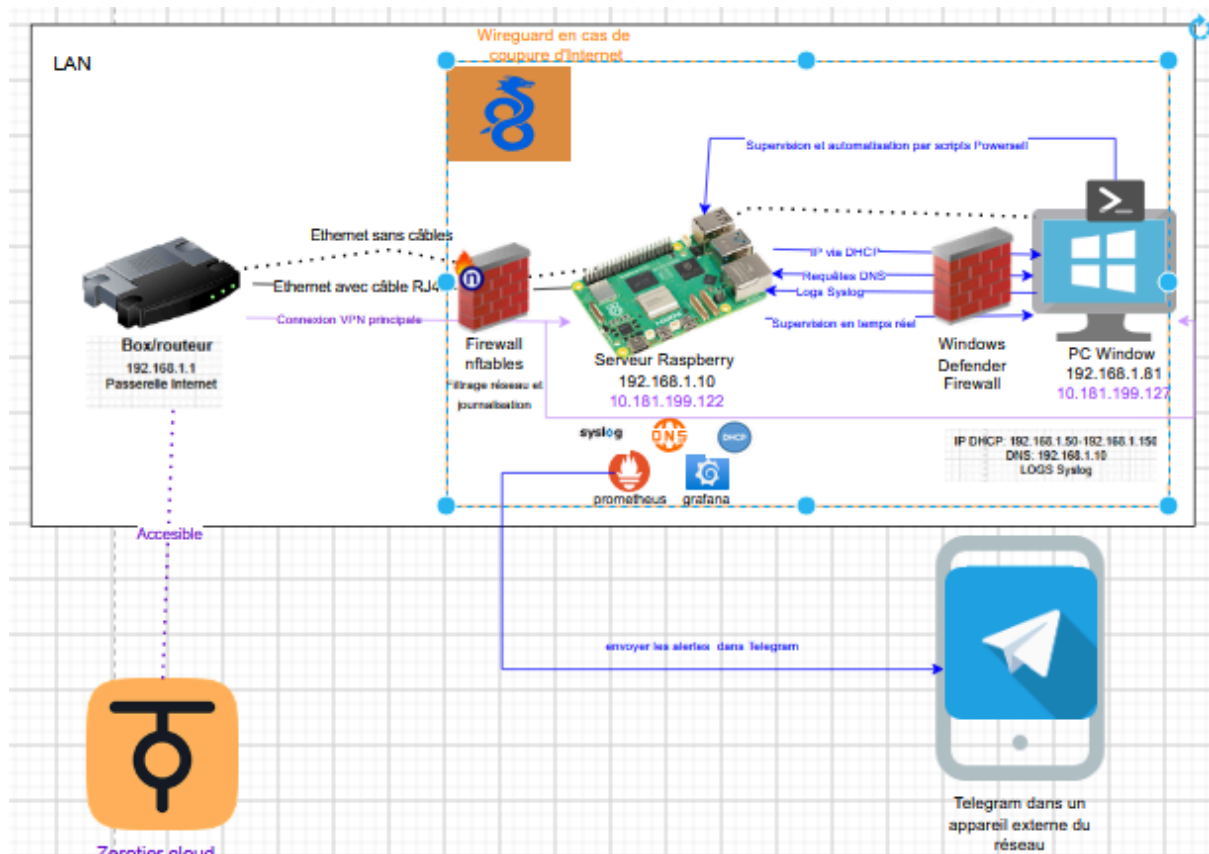
Sommaire:

1. Introduction	2
2. Présentation du projet	3
3. Travaux réalisés	4
3.1 Installation du serveur Linux	4
3.2 Supervision Prometheus / Grafana	4
3.3 Firewall nftables	10
3.4 Mise en place du VPN ZeroTier	11
3.5 Mise en place du VPN de secours WireGuard	11
3.6 Automatisation avec PowerShell	12
4. Objectifs atteints	14
5. Compétences acquises	15
Techniques	15
Réseau	15
Organisation	15
6. Problèmes rencontrés	16
7. Conclusion	17

1. Introduction

Pendant mon stage, j'ai réalisé une infrastructure informatique composée d'un serveur Linux (Raspberry Pi 5), d'un PC Windows 11 (agent), d'un système de supervision, d'un accès distant sécurisé et d'une configuration réseau stable.

Mon objectif était de créer un système simple, fonctionnel et proche d'un environnement utilisé en entreprise.

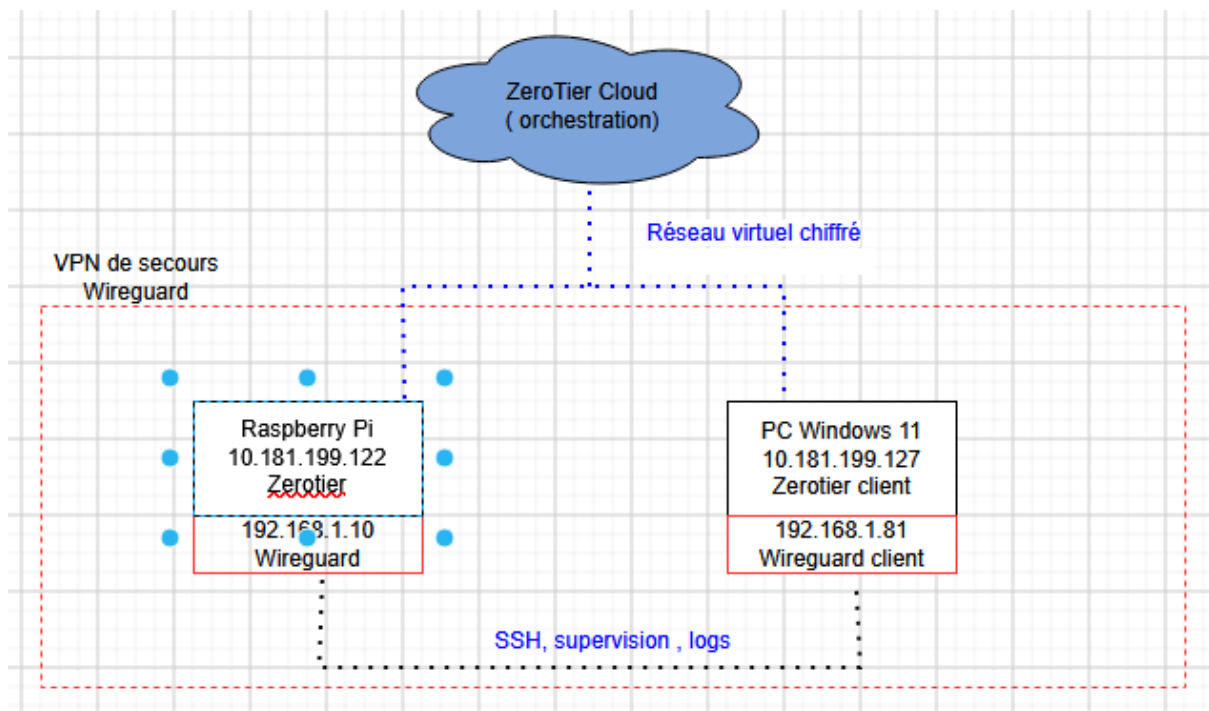


2. Présentation du projet

Mettre en place une infrastructure capable de fonctionner :

- en local,
- à distance grâce à un VPN,
- avec supervision en temps réel,
- avec une solution de secours en cas de coupure Internet.

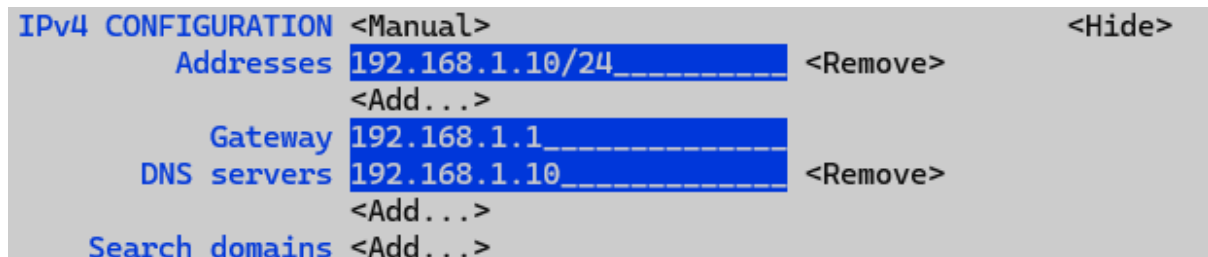
Ce projet m'a permis de comprendre comment organiser un petit système informatique complet dans le cas de VPN.



3. Travaux réalisés

3.1 Installation du serveur Linux

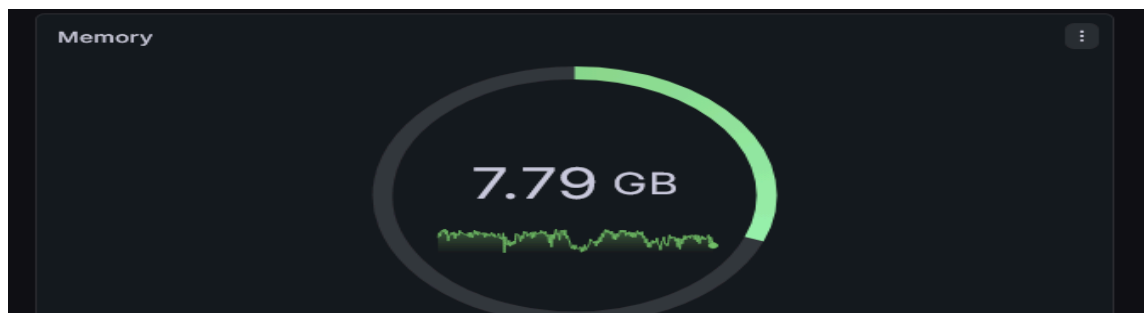
Installer Debian Bookworm (version lite) et configurer les services nécessaires pour la supervision et l'accès distant.



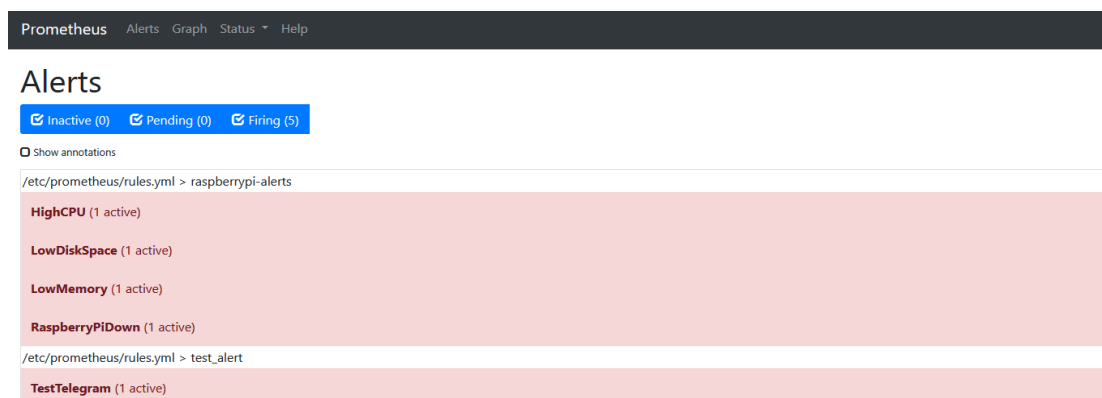
3.2 Supervision Prometheus / Grafana

Installer Prometheus et le configurer pour récupérer les métriques CPU, RAM, disque et réseau.

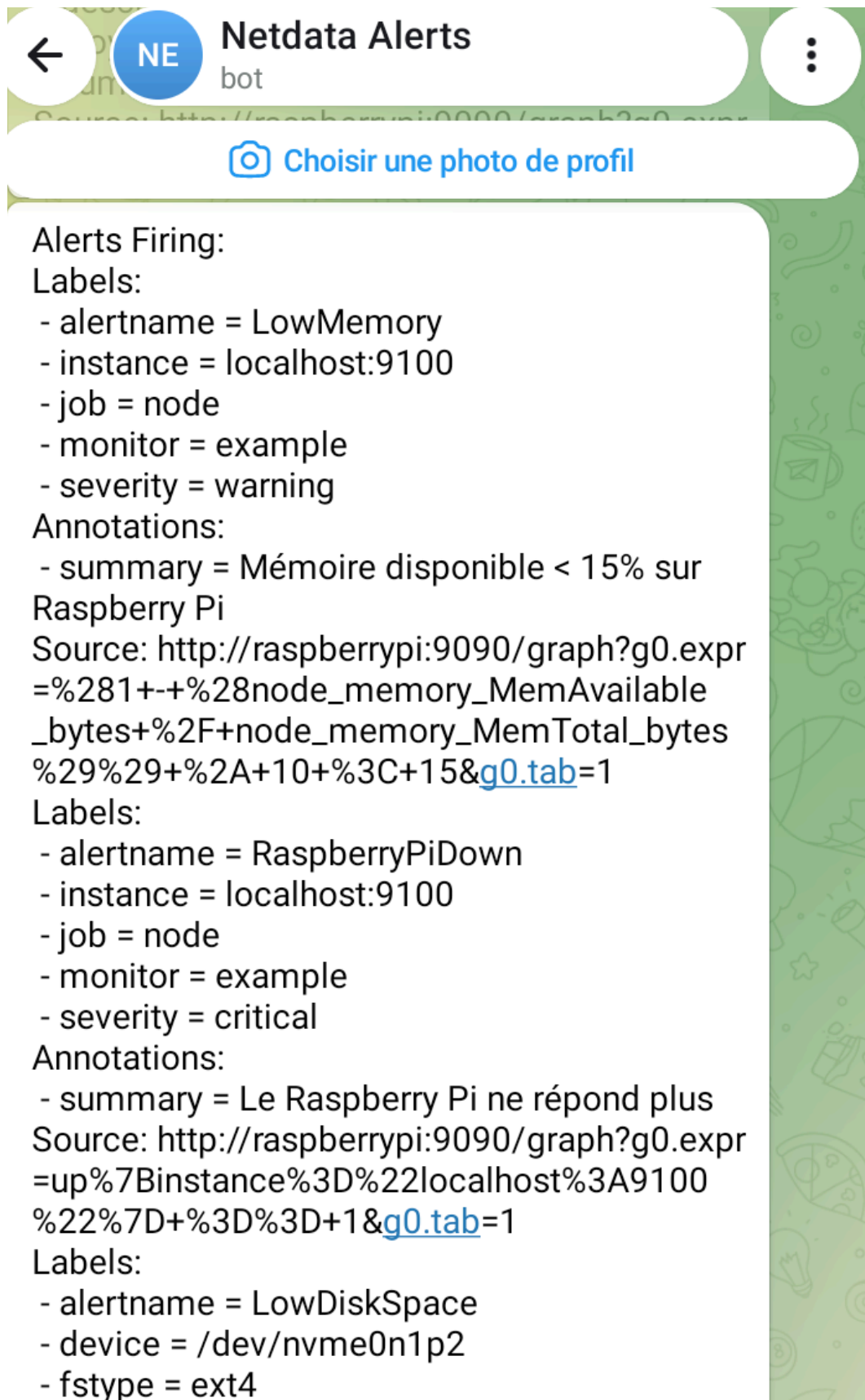
Créer des dashboards Grafana pour afficher ces données en temps réel comme par exemple un dashboard pour l'espace utilisé de la mémoire de raspberry pi 5 sur 32 Gb.



Capture : tableau des alertes actives Prometheus configurées sur le Raspberry Pi.



Capture: message envoyé par Alertmanager dans Telegram pour les alertes actives



- tstype = ext4
- instance = localhost:9100
- job = node
- monitor = example
- mountpoint = /
- severity = critical

Annotations:

- summary = Espace disque < 10% sur /
- Source: http://raspberrypi:9090/graph?g0.expr=%281+-+%28node_filesystem_avail_bytes%7Bmountpoint%3D%22%2F%22%7D+%2F+node_filesystem_size_bytes%7Bmountpoint%3D%22%2F%22%7D%29%29+%2A+100+%3C+10&g0.tab=1

Labels:

- alertname = HighCPU
- instance = localhost:9100
- monitor = example
- severity = critical

Annotations:

- summary = CPU > 80% sur Raspberry Pi
- Source: http://raspberrypi:9090/graph?g0.expr=%28avg+by+%28instance%29+%28rate%28node_cpu_seconds_total%7Bmode%3D%22idle%22%7D%5B2m%5D%29%29+%2A+100%29+%3E+80&g0.tab=1

Labels:

- alertname = TestTelegram
- monitor = example
- severity = info

Annotations:

- description = Ceci est une alerte de test envoyée depuis Alertmanager.
 - summary = Alerte de test Telegram
- Source: <http://raspberrypi:9090/graph?g0.expr=vector%281%29&g0.tab=1>

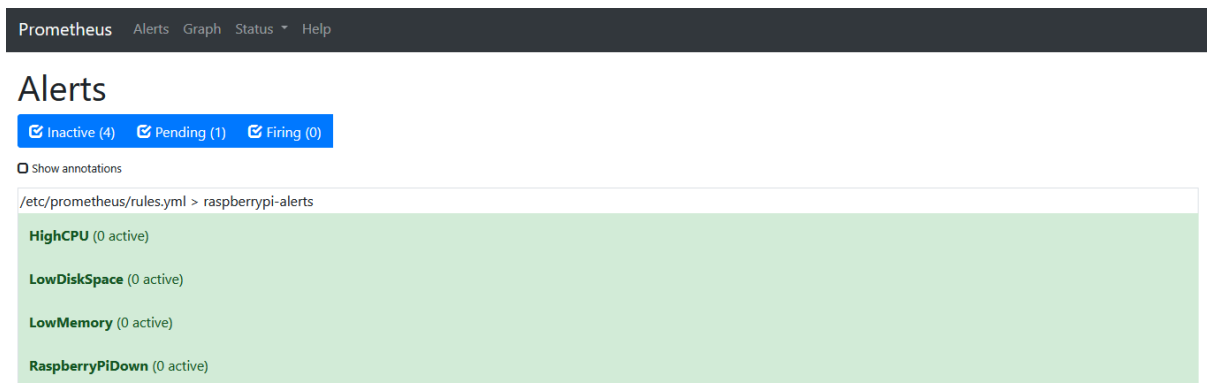
16:56



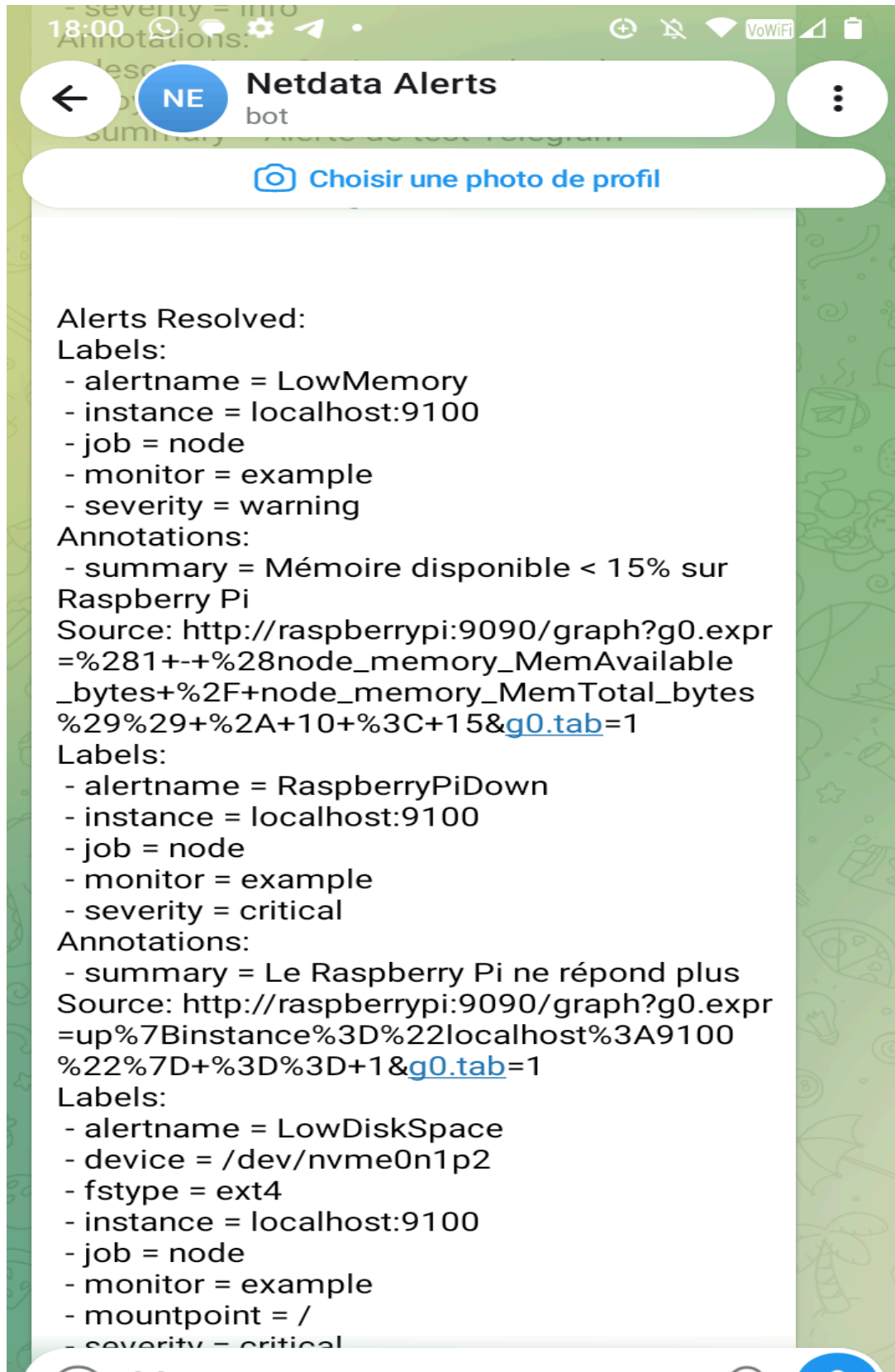
Message

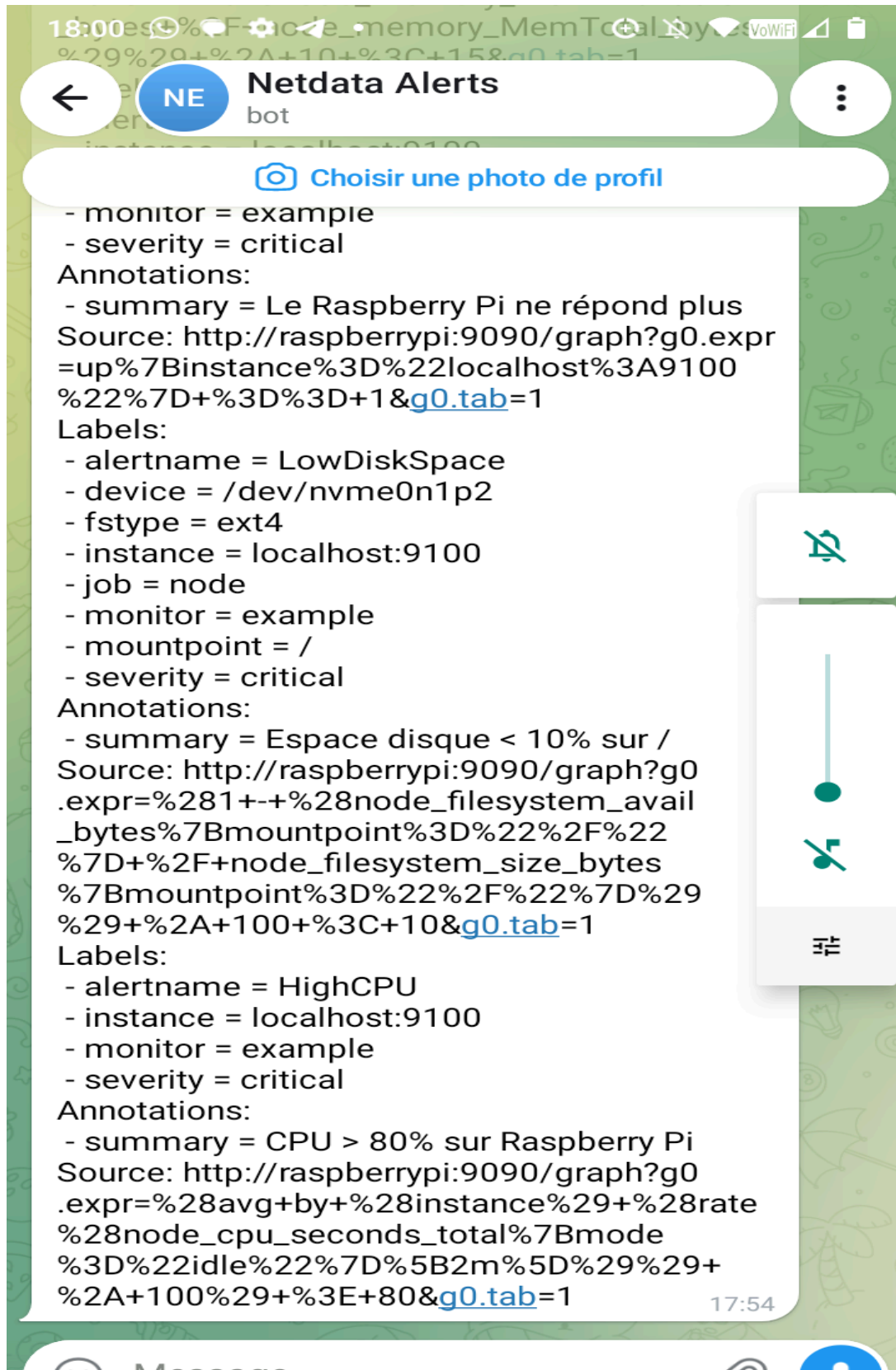


Capture : tableau des alertes résolues Prometheus configurées sur le Raspberry Pi.



Capture: message envoyé par Alertmanager dans Telegram pour les alertes résolu





3.3 Firewall nftables

J'ai installé le paquet nftables sur le serveur Linux et étudié la structure des règles.

- la création d'une table inet filter,

```
GNU nano 7.2 /etc/nftables.conf
table inet filter {
    chain input {
        type filter hook input priority filter; policy accept;
        ct state established,related accept
        iif "lo" accept
        icmp type echo-request accept
        icmp type echo-reply accept
        tcp dport 22 accept
        udp dport 53 accept
        tcp dport 53 accept
        udp dport { 67, 68 } accept
        udp dport 514 accept
        tcp dport 514 accept
        tcp dport 9090 accept
        tcp dport 9093 accept
        tcp dport 9100 accept
        tcp dport 3000 accept
        log prefix "nftables-drop: " flags all
        iif "eth0" ip saddr != 192.168.10.0/24 drop
        log prefix "NFT DROP: " flags all
        drop
        iifname "zt*" accept
    }
    chain forward {
```

- Tentatives SSH incorrectes → bannissement: `sudo fail2ban-client status sshd`

```
pi@raspberrypi:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 4
| \- File list: /var/log/auth.log
\-- Actions
    |- Currently banned: 1
    |- Total banned: 1
    \- Banned IP list: 192.168.1.81
pi@raspberrypi:~$
```

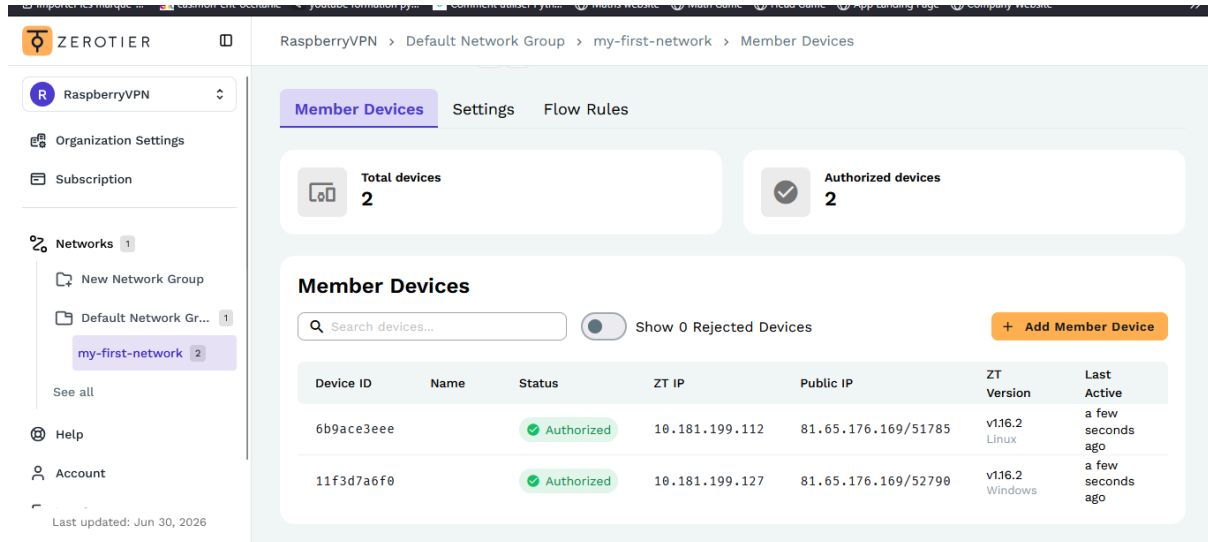
De plus, j'ai modifié certaines règles du Windows Defender Firewall afin de permettre la communication entre le serveur Linux et le PC Windows.

Ce travail m'a permis de comprendre le filtrage réseau sous Linux et pourra être complété dans une version ultérieure du projet.

3.4 Mise en place du VPN ZeroTier

Installer ZeroTier sur Linux et Windows pour permettre l'accès distant au serveur.

Capture : les deux machines (Linux et Windows) connectées au réseau ZeroTier.



Inconvénient : En cas de coupure d'Internet , Zerotier ne fonctionne plus .

Solution(hypothèse): mettre ce vpn avec 2 fournisseurs d'accès (par exemple, orange et free) donc si l'un des deux tombe , l'autre prend le relais pour éviter l'interruption de la communication de l'infrastructure informatique?

Autre Solution: un VPN de secours

3.5 Mise en place du VPN de secours WireGuard

Installer WireGuard pour assurer la continuité du réseau en cas de coupure Internet.

```
pi@raspberrypi:~$ sudo systemctl status wg-quick@wg0
● wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0
   Loaded: loaded (/lib/systemd/system/wg-quick@.service; enabled; preset: enabled)
   Active: active (exited) since Tue 2026-07-21 17:16:41 CEST; 37min ago
     Docs: man:wg-quick(8)
           https://www.wireguard.com/
           https://www.wireguard.com/quickstart/
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg-quick.8
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8
   Process: 23694 ExecStart=/usr/bin/wg-quick up wg0 (code=exited, status=0/SUCCESS)
  Main PID: 23694 (code=exited, status=0/SUCCESS)
    CPU: 30ms

Jul 21 17:16:41 raspberrypi systemd[1]: Starting wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0...
Jul 21 17:16:41 raspberrypi wg-quick[23694]: [#] ip link add wg0 type wireguard
Jul 21 17:16:41 raspberrypi wg-quick[23694]: [#] wg setconf wg0 /dev/fd/63
Jul 21 17:16:41 raspberrypi wg-quick[23694]: [#] ip -4 address add 10.143.120.1/24 dev wg0
Jul 21 17:16:41 raspberrypi wg-quick[23694]: [#] ip link set mtu 1420 up dev wg0
Jul 21 17:16:41 raspberrypi systemd[1]: Finished wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0.
```

Pour basculer de Zerotier à Wireguard automatiquement, j'ai utilisé ce code dans le raspberry pi 5 pour que le PC Window et le Serveur linux puissent se communiquer:

```
#!/bin/bash
# Vérifie si le tunnel ZeroTier est actif
ZT_STATUS=$(zerotier-cli listnetworks | grep -c "OK")

if [ "$ZT_STATUS" -eq 0 ]; then
    echo "ZeroTier inactif - activation de WireGuard..."
    systemctl start wg-quick@wg0
else
    echo "ZeroTier actif - WireGuard reste en veille."
fi
```

3.6 Automatisation avec PowerShell

Ecrire des scripts PowerShell pour automatiser :

- les diagnostics réseau,
- la vérification des services,
- l'export des résultats.

1 et 2 Capture d'écrans: script PowerShell exécuté pour récupérer les métriques du Raspberry Pi avec les résultats dans les 3 et 4 ème capture d'écrans

1) Script de check du serveur linux

```
$ip = "10.181.199.112" # IP ZeroTier du Raspberry Pi
$user = "pi"
$pass = "Parkym8Dvi7Z!"

# Commandes Linux exécutées à distance
$cpu = ssh $user@$ip "top -bn1 | grep 'Cpu(s)' | awk '{print $2}'"
$ram = ssh $user@$ip "free -m | awk '/Mem:/ {print $3}'"
$ramTotal = ssh $user@$ip "free -m | awk '/Mem:/ {print $2}'"
$disk = ssh $user@$ip "df -h / | awk 'NR==2 {print $5}'"
$temp = ssh $user@$ip "vcgencmd measure_temp | cut -d '=' -f2"
$uptime = ssh $user@$ip "uptime -p"

# Export JSON
$data = @{
    CPU = $cpu
    RAMUsedMB = $ram
    RAMTotalMB = $ramTotal
    DiskRoot = $disk
    Temperature = $temp
    Uptime = $uptime
}

$data | ConvertTo-Json | Out-File "raspberrypi.json"
```

2) Script de check de services linux

```
# =====
#   Check Services
#   =====

$ip = "10.181.199.112"
$user = "pi"

$services = @(
    "ssh",
    "prometheus",
    "grafana-server",
    "zerotier-one",
    "wireguard"
)

$result = @{}

foreach ($svc in $services) {
    $status = ssh $user@$ip "systemctl is-active $svc"
    $result[$svc] = $status
}

$result | ConvertTo-Json | Out-File "services_raspberry.json"
```

3) Script de résultat de check de serveur linux

```
{
  "CPU_Load_Percent": 43,
  "CPU_Temperature_C": "40",
  "RAM_Used_Percent": 91.67,
  "RAM_Free_GB": 0.32,
  "RAM_Total_GB": 3.82,
  "Disk_Used_Percent": 93.01,
  "Disk_Free_GB": 14.02,
  "Disk_Total_GB": 200.5,
  "Net_Sent_MB": 0,
  "Net_Recv_MB": 0,
  "Timestamp": {
    "value": "\\Date(1783495599060)\\",
    "DisplayHint": 2,
    "DateTime": "mercredi 8 juillet 2026 09:26:39"
  }
}
```

4) Script de résultat de check de services linux

```
{
  "wireguard": "active";
  "zerotier-one": "active",
  "prometheus": "active",
  "ssh": "active",
  "grafana-server": "active",
}
```

4. Objectifs atteints

- L'infrastructure fonctionne en local et à distance.
- Les métriques sont visibles en temps réel dans Grafana.
- Les scripts PowerShell automatisent les contrôles.
- *Mise en place d'un firewall nftables* pour filtrer le trafic réseau
- Configuration et utilisation du VPN ZeroTier pour l'accès distant sécurisé.
- WireGuard assure la reprise de connexion en cas de panne.
- Le serveur Linux et le PC Windows communiquent correctement.

5. Compétences acquises

Techniques

- Installation d'un serveur Linux.
- Supervision avec Prometheus et Grafana + notification de Télégram
- Compréhension des chaînes, tables, règles et journalisation avec nftables
- Mise en place d'un VPN ZeroTier.
- Automatisation avec PowerShell.
- Gestion des services Linux.
- Analyse et correction de pannes.

Réseau

- Filtrage réseau avec nftables : règles d'accès, ports autorisés, logs.
- Mise en place de VPN (ZeroTier + WireGuard).
- Configuration des interfaces réseau.
- Gestion des routes et des passerelles.
- Résolution de conflits réseau.

Organisation

- Structuration d'un mini-système d'information.
- Documentation claire.
- Travail autonome.

6. Problèmes rencontrés

- Problème de clé SSH → résolu avec `ssh-keygen -R`.
- Difficulté à envoyer des alertes Prometheus → correction de la configuration Alertmanager pour envoyer ces alertes dans la messagerie de Telegram.
- Perte du tunnel ZeroTier (voir les détails dans 3.4) → mise en place de WireGuard comme secours.

Capture : correction de la configuration Alertmanager.

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 1h
  receiver: 'telegram'

receivers:
- name: 'telegram'
  telegram_configs:
    - bot_token: "7909503446:AAHx8R8Kmy9H4m3MDaoN4utOstkLie1r7wY"
      chat_id: 1713123302
      message: "⚠️ Alerte Prometheus : {{ .CommonAnnotations.summary }}"

inhibit_rules:
- source_match:
    severity: 'critical'
  target_match:
    severity: 'warning'
  equal: ['alertname', 'dev', 'instance']
```

7. Conclusion

Ce stage m'a permis de créer une infrastructure complète, stable, supervisée et sécurisée.

J'ai appris à installer, superviser, sécuriser et automatiser un système informatique tout en développant mes compétences en réseau et en administration système.

Ce projet m'a apporté une expérience concrète et utile pour la suite de mon parcours.